

CHƯƠNG 2 - DOCKERFILE VÀ TỰ TẠO IMAGE

BÀI 6

SỬA SOURCE, REBUILD IMAGE VÀ BUILD CACHE

Bài học tập trung vào mối quan hệ giữa Source Code, Docker Image và Container khi ứng dụng được cập nhật, đồng thời làm rõ cơ chế Build Cache.

1. Mục tiêu bài học

- Hiểu sửa source code không làm Image tự thay đổi.
- Hiểu rebuild Image không làm Container cũ tự cập nhật.
- Biết vì sao cần tạo lại Container để sử dụng Image mới.
- Hiểu Build Cache hoạt động khi dữ liệu đầu vào của bước build không thay đổi.
- Biết khi nào bước COPY phải chạy lại.
- Phân biệt docker build, docker run và docker start trong quá trình cập nhật ứng dụng.
- Thực hiện được chuỗi cập nhật source -> rebuild Image -> recreate Container.

2. Trạng thái ban đầu

Project hiện tại đã hoàn thành chuỗi:

```
index.html
↓
Dockerfile
↓ docker build
my-web:1.0
↓ docker run
my-web-container
↓
localhost:8080
```

Website đang chạy trong Container từ Image được build ở phiên bản trước.

3. Sửa source code

Mở file:

```
E:\Docker\Projects\my-web\index.html
```

Ví dụ sửa nội dung thành phiên bản mới:

```
Hello Docker Version 2!
Tôi vừa cập nhật source code của website.
```

Sau khi save, chỉ có source code trên máy Windows thay đổi.

```
Source Code: Version 2
Image: Version 1
Container: Version 1
```

Ghi nhớ: Sửa source code không làm Image và Container tự thay đổi.

4. Vì sao reload browser vẫn thấy nội dung cũ?

Container không trực tiếp đọc file index.html trong thư mục project trên Windows. File đã được COPY vào Image trong lần build trước.

```
Windows:
E:\Docker\Projects\my-web\index.html
Version 2

Container:
/usr/share/nginx/html/index.html
Version 1
```

Do đó reload browser trước khi rebuild vẫn hiển thị phiên bản cũ.

5. Rebuild Image

Để Image phản ánh source mới, chạy:

```
cd E:\Docker\Projects\my-web
docker build -t my-web:1.0 .
```

Docker đọc lại Dockerfile và tạo kết quả Image mới dựa trên source hiện tại.

```
Source Version 2
↓
docker build
↓
my-web:1.0
↓
Image phản ánh Version 2
```

6. Build Cache hoạt động thế nào?

Docker cố gắng tái sử dụng kết quả của các bước build trước nếu instruction và dữ liệu đầu vào liên quan không thay đổi.

Dockerfile của project:

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html
```

Trường hợp FROM

Nếu Base Image và bước FROM không thay đổi, Docker có thể tái sử dụng dữ liệu/cache phù hợp.

Trường hợp COPY

Nếu index.html thay đổi thì đầu vào của COPY thay đổi. Cache cũ của bước COPY không còn phù hợp, vì vậy Docker phải thực hiện lại COPY.

```
index.html không đổi
↓
COPY có thể CACHED

index.html thay đổi
↓
cache COPY không còn phù hợp
↓
COPY chạy lại
```

Ghi nhớ: Build Cache không có nghĩa Docker luôn dùng cache. Cache chỉ được tái sử dụng khi bước build và dữ liệu đầu vào vẫn phù hợp.

7. Build lại mà không sửa gì

Sau khi đã build Version 2, nếu không sửa Dockerfile hoặc index.html và chạy lại:

```
docker build -t my-web:1.0 .
```

Docker có thể hiển thị:

```
CACHED
```

ở các bước không thay đổi.

Lợi ích:

- Không thực hiện lại công việc không cần thiết.
- Giảm thời gian build.
- Tái sử dụng kết quả build trước đó.

8. Rebuild Image có cập nhật Container cũ không?

Không. Đây là ý quan trọng nhất của bài.

```
Image V1
↓ docker run
Container A
```

```
Sau đó:
Source V2
↓ docker build
Image V2
```

Container A vẫn là Container cũ

Container đã được tạo từ Image tại thời điểm docker run trước đó. Việc tạo một Image mới không làm Container đã tồn tại tự chuyển sang Image mới.

Lưu ý: Lý do không phải vì Container đang chạy. Ngay cả khi stop rồi start, nó vẫn là chính Container cũ.

9. docker start không tạo lại Container

```
docker stop my-web-container
docker start my-web-container
```

Hai lệnh này chỉ dừng và chạy lại cùng một Container đã tồn tại.

```
docker start
→ chạy lại Container cũ

docker run
→ tạo Container mới từ Image
```

Vì vậy docker start không làm Container cũ chuyển sang Image vừa rebuild.

10. Muốn Container dùng Image mới

Quy trình rõ ràng trong bài này:

```
Sở dĩ Source
↓
docker build lại
↓
Image mới
↓
docker stop Container cũ
↓
docker rm Container cũ
↓
docker run
↓
Container mới từ Image mới
```

Các lệnh:

```
docker build -t my-web:1.0 .

docker stop my-web-container

docker rm my-web-container

docker run -d --name my-web-container -p 8080:80 my-web:1.0
```

Ghi nhớ: docker build tạo Image. docker run mới tạo Container.

11. Ba trạng thái cần phân biệt

Tình huống A - Chỉ sửa source

```
Source: V2
Image: V1
Container: V1
```

Image và Container chưa thay đổi.

Tình huống B - Sửa source và rebuild

```
Source: V2
Image: V2
Container: V1
```

Image đã mới nhưng Container cũ vẫn chưa thay đổi.

Tình huống C - Sửa, rebuild và recreate Container

```
Source: V2
Image: V2
Container: V2
```

Đây là trạng thái đồng bộ.

12. Chuỗi Docker tổng quát

```
Source Code
↓
Dockerfile
↓
docker build
↓
Image
↓
docker run
↓
Container
```

Khi cập nhật ứng dụng:

```
Sửa Source
↓
docker build lại
↓
Image mới
↓
recreate Container
↓
Container mới
```

13. Thực hành đề xuất

Bài tập thực hành hoàn chỉnh:

1. `docker ps`
2. Sửa `index.html` thành `Version 2`
3. Reload browser trước khi build
4. `docker build -t my-web:1.0 .`
5. Reload browser khi Container cũ vẫn chạy
6. `docker stop my-web-container`
7. `docker rm my-web-container`
8. `docker run -d --name my-web-container -p 8080:80 my-web:1.0`
9. Mở `http://localhost:8080`
10. Không sửa gì và `docker build -t my-web:1.0 .` lại nữa
11. Quan sát `CACHED`

14. Câu hỏi kiểm tra và đáp án

Câu 1. Sửa index.html nhưng chưa docker build, Image có thay đổi không?

Không. Source code thay đổi nhưng Image vẫn là kết quả của lần build trước.

Câu 2. Build lại my-web:1.0 thì Container đang chạy có tự cập nhật không?

Không. Container cũ không tự chuyển sang Image mới.

Câu 3. Vì sao Container cũ không tự dùng Image mới?

Vì Container đã được tạo từ Image tại thời điểm docker run trước đó. Rebuild một Image không thay đổi Container đã tồn tại.

Câu 4. docker start có tạo lại Container từ Image mới không?

Không. docker start chỉ khởi động lại Container đã tồn tại.

Câu 5. Muốn Container dùng Image mới phải làm gì?

Rebuild Image, dừng và xóa Container cũ, sau đó docker run để tạo Container mới từ Image mới.

Câu 6. Nếu index.html thay đổi thì bước COPY có dùng cache cũ không?

Không. Vì đầu vào của COPY đã thay đổi nên cache cũ của bước đó không còn phù hợp.

Câu 7. Nếu build lại mà không sửa gì thì Build Cache giúp gì?

Docker có thể tái sử dụng kết quả các bước không thay đổi để build nhanh hơn.

Câu 8. Chuỗi nào đúng?

Source -> Dockerfile -> docker build -> Image -> docker run -> Container.

15. Chỉnh lại các câu trả lời dễ nhầm

Không phải: 'Container không cập nhật vì nó đang chạy'

Container không cập nhật vì nó đã được tạo từ Image cũ. Trạng thái Running hay Exited không làm nó tự chuyển sang Image mới.

Không phải: 'xóa Container rồi docker build để tạo Container mới'

docker build chỉ tạo Image. Sau khi build, cần docker run để tạo Container mới.

Không phải: 'Build Cache luôn được dùng'

Cache chỉ được dùng khi Docker xác định bước build và dữ liệu liên quan chưa thay đổi.

16. Bài tập

Bài tập 1 - Cache:

- Build project khi index.html ở Version 2.
- Không sửa gì, build lại và ghi lại bước nào hiện CACHED.
- Sửa một dòng trong index.html rồi build lại và quan sát bước COPY.

Bài tập 2 - Container cũ:

- Build Image mới nhưng chưa recreate Container.
- Reload browser và ghi lại nội dung đang hiển thị.
- Giải thích vì sao Container vẫn dùng nội dung cũ.

Bài tập 3 - Recreate:

- Stop và rm Container cũ.

- Run Container mới từ my-web:1.0.
- Kiểm tra browser và xác nhận Version 2.

Bài tập 4 - Tự giải thích:

```
docker build -t my-web:1.0 .  
docker stop my-web-container  
docker rm my-web-container  
docker run -d --name my-web-container -p 8080:80 my-web:1.0
```

Giải thích từng lệnh đang tác động lên Source, Image hay Container.

17. Tóm tắt cú pháp và câu lệnh

Build/Rebuild Image

```
docker build -t my-web:1.0 .
```

Tạo hoặc cập nhật kết quả Image từ Dockerfile và source hiện tại.

Kiểm tra Image

```
docker images
```

Kiểm tra Container đang chạy

```
docker ps
```

Dừng Container cũ

```
docker stop my-web-container
```

Xóa Container cũ

```
docker rm my-web-container
```

Tạo Container mới từ Image

```
docker run -d --name my-web-container -p 8080:80 my-web:1.0
```

Khởi động lại Container cũ

```
docker start my-web-container
```

Chỉ dùng khi muốn chạy lại cùng Container, không phải để cập nhật sang Image mới.

18. Thuật ngữ cuối bài

Rebuild

Build lại Docker Image sau khi source hoặc Dockerfile thay đổi.

Build Cache

Cơ chế tái sử dụng kết quả của các bước build không thay đổi.

Cache Hit

Trường hợp Docker tìm thấy cache phù hợp và có thể tái sử dụng.

Cache Invalidation

Trường hợp dữ liệu/instruction thay đổi khiến cache cũ không còn phù hợp.

CACHED

Dòng output cho biết Docker đang tái sử dụng kết quả build trước.

Recreate Container

Xóa Container cũ và tạo Container mới từ Image hiện tại.

Source Code

File mã nguồn/application file trên máy phát triển, ví dụ index.html.

Image

Artifact được build từ Dockerfile và source, dùng làm mẫu tạo Container.

Container

Instance được tạo từ Image tại thời điểm docker run.

docker build

Tạo Image.

docker run

Tạo Container mới từ Image rồi chạy.

docker start

Khởi động lại Container đã tồn tại.

COPY

Instruction copy file từ Build Context vào Image.

Build Context

Tập hợp file Docker Build có thể sử dụng; trong docker build . là thư mục hiện tại.

19. Sơ đồ tổng kết

```
SOURCE V2
index.html
↓
docker build
↓
IMAGE V2
my-web:1.0
↓
stop + rm Container cũ
↓
docker run
↓
CONTAINER V2
my-web-container
↓
localhost:8080
```

Ghi nhớ: Điểm quan trọng nhất: Source, Image và Container là ba trạng thái tách biệt. Thay đổi Source không tự đổi Image; thay đổi Image không tự đổi Container.

Bài tiếp theo: Chương 2 - Bài 7: WORKDIR, RUN, CMD, ENTRYPOINT và EXPOSE